Open in app ↗

Medium

Access to everything. Now 30% off. Upgrade now

From Raw Text to Intelligent Predictions: Understanding the Complete NLP Workflow

12 min read · Just now

Bhuvaneshwari

Listen Share ••• More

Introduction

Every day, we generate enormous amounts of textual data through news articles, social media posts, emails, customer reviews, online discussions, and messages.

But how can a computer understand this text? How can it identify whether a news article is about business, politics, sports, entertainment, or technology?

This is where **Natural Language Processing (NLP)** comes into the picture.

Natural Language Processing is a branch of Artificial Intelligence that enables computers to process, analyze, interpret, and extract meaningful information from human language.

NLP combines linguistic concepts, machine learning, and statistical techniques to transform unstructured text into meaningful insights and predictions.

Some real-world applications include:

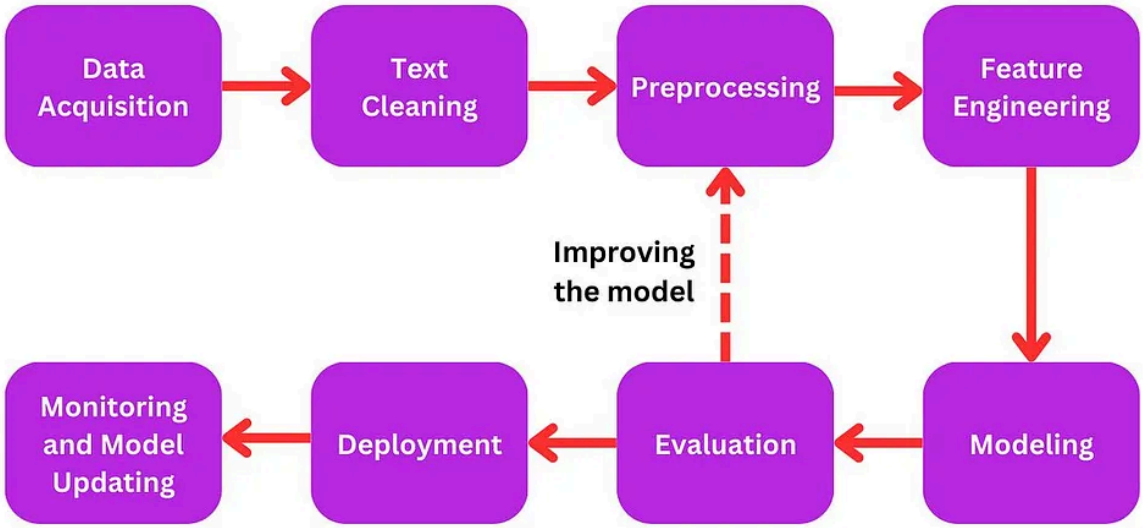
- **Spam Detection:** Identifying unwanted emails and messages.
- **Sentiment Analysis:** Understanding whether customer feedback is positive, negative, or neutral.
- **News Classification:** Automatically organizing news articles into categories.

- **Chatbots:** Understanding user questions and generating appropriate responses.
- **Language Translation:** Translating text from one language into another.
- **Text Summarization:** Generating shorter versions of lengthy documents.

However, building an NLP application involves much more than simply training a machine learning model. It requires a systematic process called the **NLP Workflow**.

Let's explore each stage step by step.

1. Understanding the NLP Workflow



An NLP workflow is a sequence of steps that transforms raw textual data into useful information or machine learning predictions.

The general workflow looks like this:

Raw Text → Data Collection → Data Exploration → Text Cleaning → Preprocessing → Feature Extraction → Model Training → Evaluation → Prediction → Deployment

Each stage contributes to the quality and reliability of the final application.

2. Data Collection

The first step is collecting the textual data required to solve the problem.

Depending on the application, data can come from:

- CSV and Excel files
- Text documents and PDFs
- Websites and web scraping
- APIs
- Social media platforms
- Customer reviews and feedback
- Public datasets

Example: News Article Classification

Suppose we want to develop a system that automatically classifies news articles.

Our dataset may contain two important columns:

News Article	Category
The stock market experienced significant growth...	Business
The national team won the championship...	Sport
New smartphone technology was introduced...	Tech
The government announced new policies...	Politics

Here:

- News Article is the input text.
- Category is the target label.

The objective is to train a model that learns the relationship between article content and its category.

3. Data Understanding and Exploration

After collecting the data, we need to understand its structure and quality.

This stage is known as Exploratory Data Analysis (EDA).

Important activities include:

- Checking dataset dimensions.
- Understanding column names and data types.
- Identifying missing values.
- Finding duplicate records.
- Examining category distribution.
- Analyzing text length and word frequency.

Python implementation:

```
import pandas as pd
```

```
df = pd.read_csv("bbc_news.csv", encoding="latin-1")

print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
```

Why is EDA important?

EDA helps us identify problems before preprocessing and model development.

For example, if one category contains significantly more articles than another, the model may become biased toward the majority category.

Understanding these patterns helps us choose appropriate preprocessing and evaluation techniques.

4. Text Cleaning

Raw textual data often contains unnecessary elements that may affect analysis.

Consider this example:

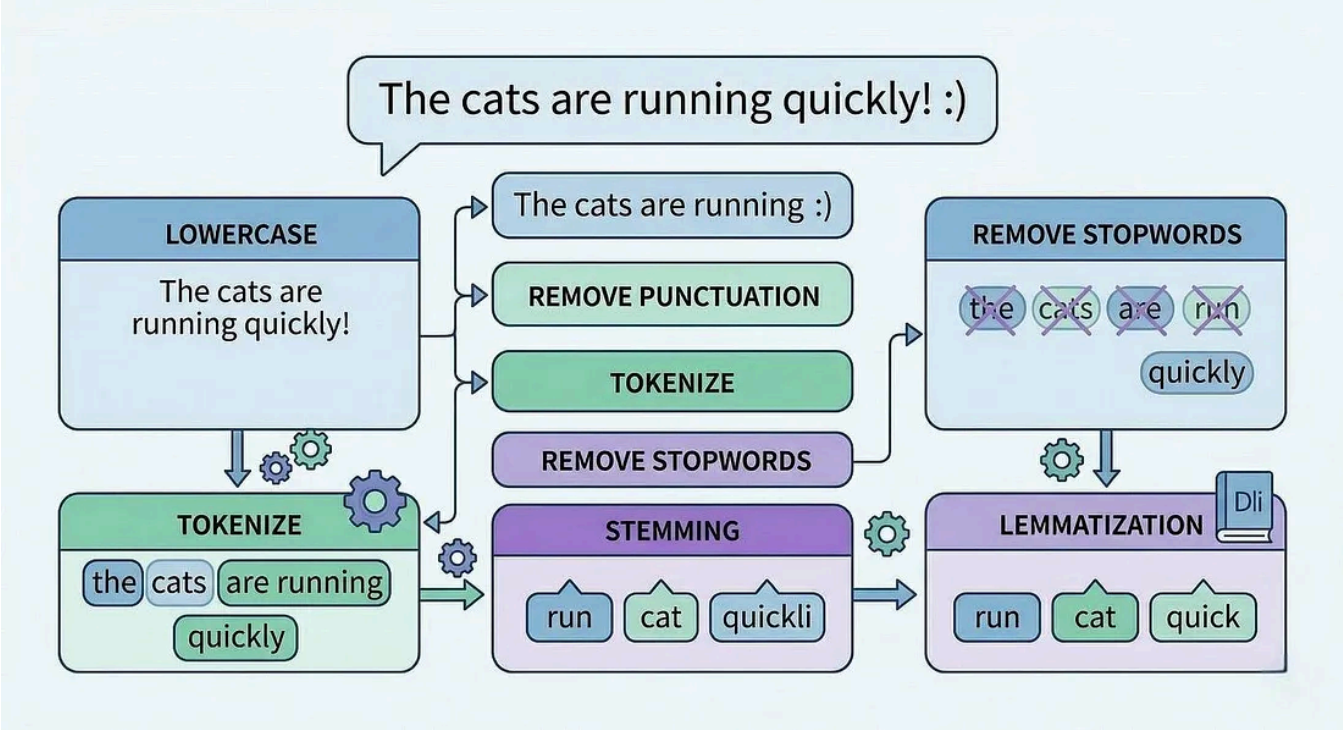
Original text:

“BREAKING NEWS!!! Visit https://example.com for the LATEST updates about Technology in 2026.”

After text cleaning:

“breaking news visit for the latest updates about technology in”

The objective is to remove irrelevant characters and standardize the text.



Common text-cleaning operations include:

4.1 Lowercasing

Converts all text into lowercase.

Example:

```
"Machine Learning Is Powerful"
```

Becomes:

```
"machine learning is powerful"
```

This prevents words with different capitalization from being treated as separate terms.

4.2 Removing URLs

URLs generally do not contribute useful information in many text classification tasks.

Example:

```
"Visit https://example.com today"
```

Becomes:

```
"Visit today"
```

4.3 Removing HTML Tags

Web-extracted text may contain HTML elements.

Example:

```
"<h1>Welcome to NLP</h1>"
```

Becomes:

```
"Welcome to NLP"
```

4.4 Removing Punctuation and Special Characters

Example:

```
"NLP is amazing!!!"
```

Becomes:

```
"NLP is amazing"
```

4.5 Removing Extra Spaces

Multiple spaces are replaced with a single space.

Python implementation:

```
import re
```

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text)

    return text.strip()
```

Text cleaning creates a more consistent representation of the original content.

Note: Cleaning rules should depend on the problem. For example, punctuation, numbers, or special characters may carry useful information in sentiment analysis or financial text.

5. Tokenization

Tokenization is the process of dividing text into smaller units called tokens.

These tokens can be words, sentences, or subwords.

For basic word tokenization, consider:

Input:

```
"Natural Language Processing is interesting"
```

Output:

```
["natural", "language", "processing", "is", "interesting"]
```

Python implementation:

```
text = "Natural Language Processing is interesting"
```

```
tokens = text.lower().split()
print(tokens)
```

Tokenization allows us to analyze individual words and prepare text for subsequent NLP operations.

6. Stopword Removal

Stopwords are commonly occurring words that may contribute limited information to certain NLP tasks.

Examples include:

- is
- the
- a
- an
- in
- of
- and
- are

For many text classification problems, removing these words can reduce unnecessary features.

Example:

Before:

```
"The machine learning model is working on the data"
```

After:

```
"machine learning model working data"
```

Python implementation:

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
```

```
def remove_stopwords(tokens):
    return [
        word for word in tokens
        if word not in ENGLISH_STOP_WORDS
    ]
```

Important: Stopword removal is not always necessary. Words such as “not” can change the meaning of a sentence, especially in sentiment analysis. Always consider the problem before removing them.

7. Stemming and Lemmatization

Stemming and lemmatization are techniques used to reduce words to a common base or root representation.

7.1 Stemming

Stemming removes word endings using predefined rules.

Examples:

Original Word	Stemmed Word
Playing	Play
Connected	Connect
Studies	Studi
Running	Run

Stemming is generally faster but may produce words that are not grammatically correct.

7.2 Lemmatization

Lemmatization converts words into their meaningful dictionary form, called a lemma.

Examples:

Original Word	Lemmatized Word
Running	Run
Better	Good
Studies	Study
Cars	Car

Lemmatization considers vocabulary and sometimes grammatical context, making it more linguistically meaningful.

Stemming	Lemmatization
Uses rule-based word reduction	Uses vocabulary and linguistic analysis
Usually faster	Usually more computationally expensive
May produce incomplete words	Produces meaningful base words
Less linguistically accurate	Generally more linguistically accurate

Stemming vs. Lemmatization

The choice depends on the dataset, application, and performance requirements.

8. Feature Extraction: Converting Text into Numbers

Machine learning algorithms cannot directly understand raw textual data.

They require numerical representations.

Therefore, we need to convert text into numerical features. This process is called **Feature Extraction** or **Text Vectorization**.

Common techniques include:

1. Bag of Words (BoW)
2. Term Frequency–Inverse Document Frequency (TF-IDF)

3. Word Embeddings

8.1 Bag of Words (BoW)

Bag of Words represents documents based on the frequency of words appearing in them.

Consider two sentences:

- Document 1: “Machine learning is useful”
- Document 2: “Machine learning is powerful”

Vocabulary:

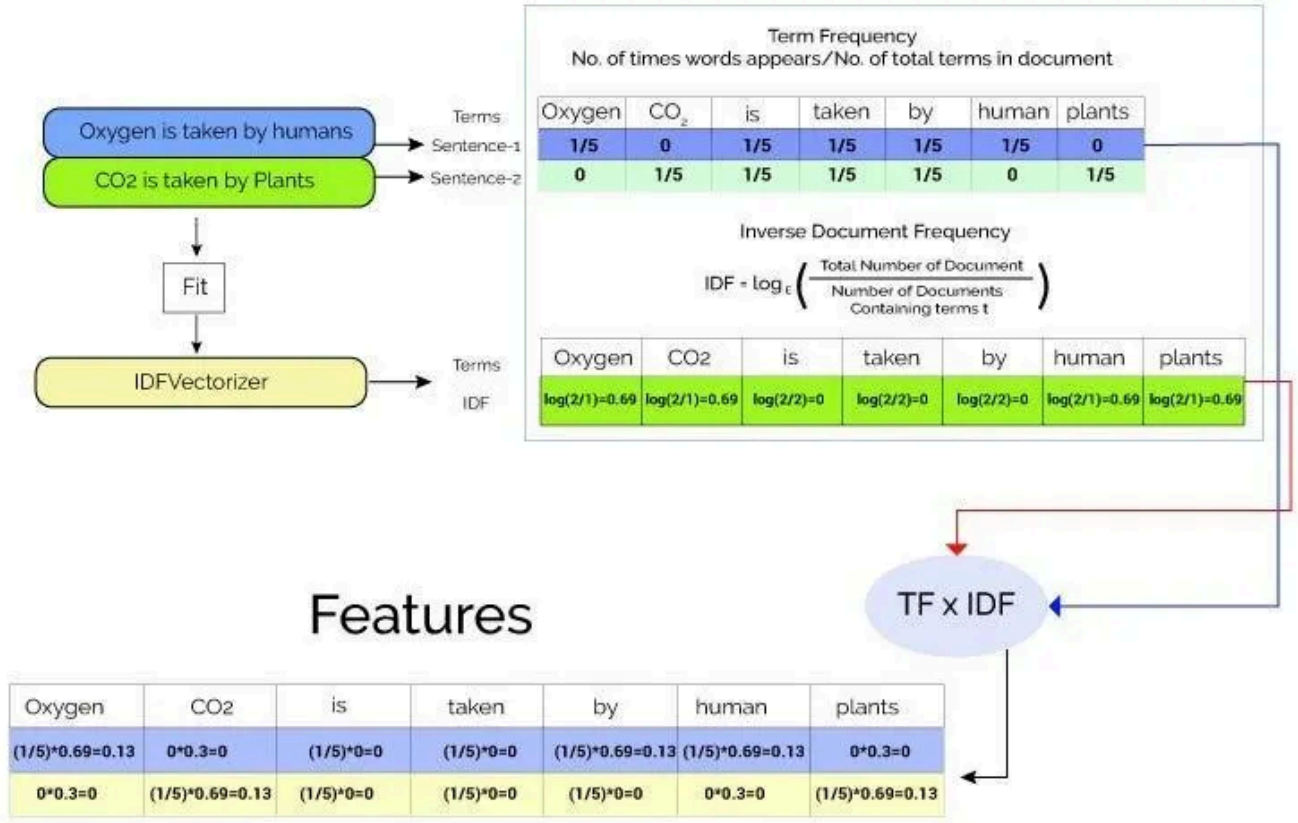
```
["machine", "learning", "is", "useful", "powerful"]
```

The documents can be represented numerically:

Document	Machine	Learning	Is	Useful	Powerful
Document 1	1	1	1	1	0
Document 2	1	1	1	0	1

BoW is simple and useful for many basic text classification problems, but it does not capture the importance of individual words or their context.

8.2 TF-IDF



TF-IDF stands for Term Frequency–Inverse Document Frequency.

It measures how important a word is within a document relative to the entire collection of documents.

It combines two components:

Term Frequency (TF): Measures how frequently a word appears in a document.

Inverse Document Frequency (IDF): Reduces the importance of words that appear in many documents.

The general formula is:

TF-IDF = TF × IDF

A word that appears frequently in a particular document but less frequently across the entire dataset can receive a higher TF-IDF score.

For example, in a collection of news articles, words such as “government”, “football”, and “technology” may help distinguish different topics.

Python implementation:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf_vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 1)
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)

X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

Notice that we use `fit_transform()` on training data and only `transform()` on testing data.

This prevents information from the test set from influencing feature vocabulary and IDF values during training.

8.3 Word Embeddings

Word embeddings represent words as dense numerical vectors.

Unlike traditional methods, they can capture semantic relationships between words.



Popular approaches include:

- Word2Vec
- GloVe

- FastText
- Contextual embeddings from transformer models

For more advanced NLP applications, embeddings can provide richer representations of language.

9. Train-Test Split

After preparing the text data and selecting the target labels, we divide the dataset into training and testing sets.

Training data: Used to teach the model.

Testing data: Used to evaluate how well the model performs on unseen examples.

A common split is:

- 80% Training Data
- 20% Testing Data

Python implementation:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Stratification helps preserve the category proportions in both datasets.

Important: Split the data before fitting the TF-IDF vectorizer. This helps prevent data leakage.

10. Model Building and Training

Once the text is converted into numerical features, we can train machine learning algorithms.

The model learns patterns from the training data and uses those patterns to classify new documents.

For text classification, commonly used algorithms include:

- Multinomial Naive Bayes
- Logistic Regression
- Support Vector Machines (SVM)
- Decision Trees
- Random Forest

Let's understand two commonly used models.

10.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem.

It is commonly used for text classification because it works well with word-frequency and TF-IDF-style features.

Applications include:

- Spam detection
- News classification
- Document categorization

10.2 Logistic Regression

Despite its name, Logistic Regression is widely used for classification tasks.

It estimates the probability that an input belongs to a particular class.

For multi-class classification, it can use strategies such as multinomial classification to distinguish between multiple categories.

It is useful for:

- Sentiment analysis
- News classification
- Email categorization
- Document classification

Python implementation:

```
from sklearn.naive_bayes import MultinomialNB
```

```
news_model = MultinomialNB()
news_model.fit(X_train_tfidf, y_train)
```

After training and evaluating the first model, we can train another model:

```
from sklearn.linear_model import LogisticRegression
```

```
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_tfidf, y_train)
```

Training models separately makes it easier to understand their performance and compare their results.

11. Model Evaluation

Training a model is not enough. We must measure how accurately it predicts unseen data.

Several evaluation metrics help us understand model performance.

11.1 Accuracy

Accuracy measures the proportion of correctly predicted observations.

Accuracy = Correct Predictions / Total Predictions

A higher accuracy indicates that more predictions are correct.

11.2 Precision

Precision measures how many predicted positive cases are actually positive.

It is useful when false positive predictions are costly.

11.3 Recall

Recall measures how many actual positive cases the model correctly identifies.

It is important when missing a relevant class is costly.

11.4 F1-Score

F1-score is the harmonic mean of precision and recall.

It provides a combined measure of both metrics.

11.5 Confusion Matrix

A confusion matrix shows how many observations were correctly and incorrectly classified for each category.

For a multi-class news classification problem, it helps identify which categories the model confuses with one another.

Python implementation:

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

```
y_pred = news_model.predict(X_test_tfidf)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))

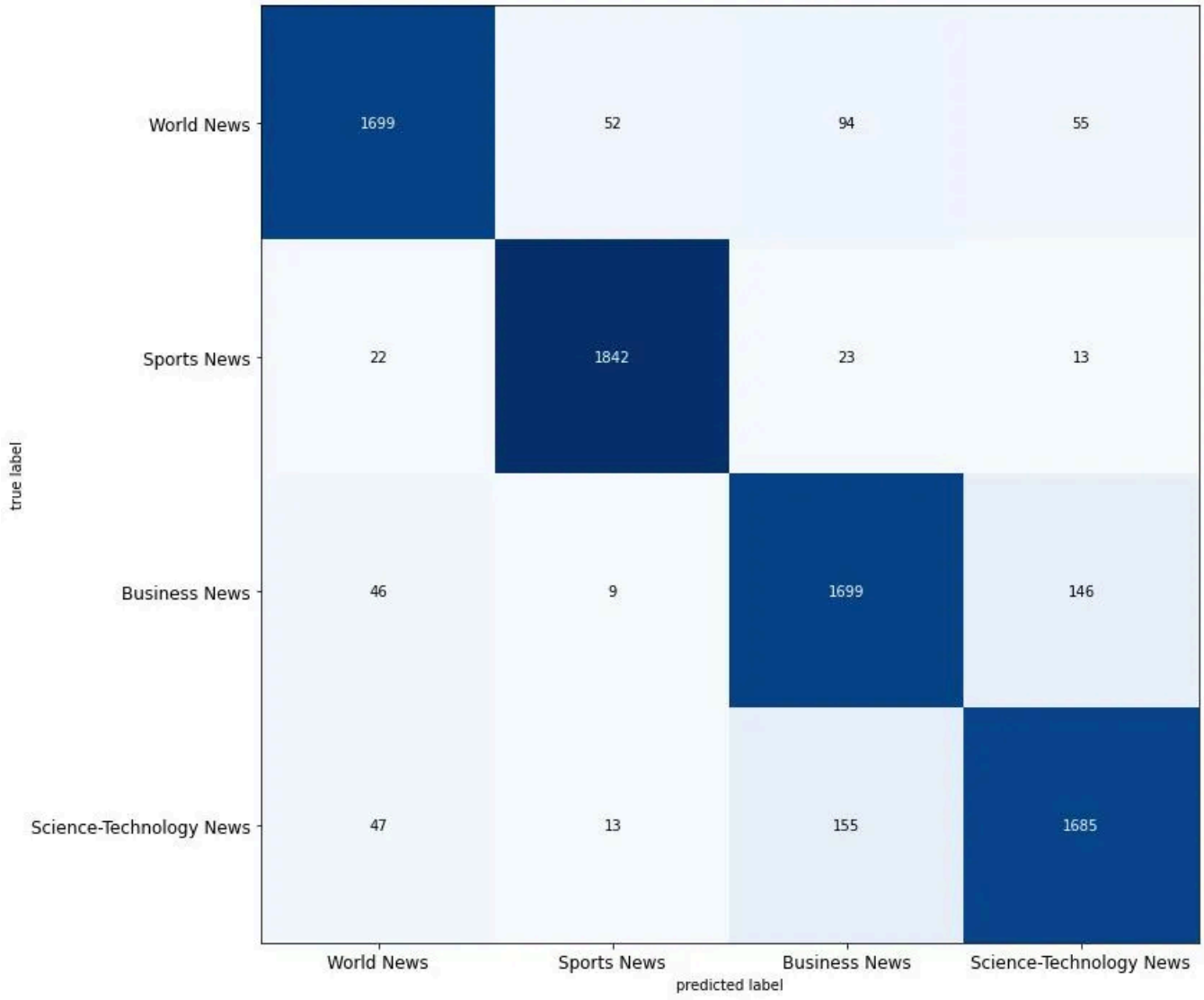
print(
    "Precision:",
    precision_score(
        y_test, y_pred,
        average="weighted",
        zero_division=0
    )
)

print(
    "Recall:",
    recall_score(
        y_test, y_pred,
        average="weighted",
        zero_division=0
    )
)

print(
    "F1 Score:",
    f1_score(
        y_test, y_pred,
        average="weighted",
        zero_division=0
    )
)

print(classification_report(y_test, y_pred))
```

Using multiple metrics gives a more complete understanding of model performance than relying on accuracy alone.



12. Model Comparison

When multiple algorithms are trained, their performance should be compared using the same test data and evaluation metrics.

For example:

Evaluation Metric	Naive Bayes	Logistic Regression
Accuracy	Compare	Compare
Precision	Compare	Compare
Recall	Compare	Compare
F1-Score	Compare	Compare

The best model should be selected based on the problem's requirements and the evaluation results.

A model with higher accuracy is not automatically the best choice in every situation. Precision, recall, F1-score, computational cost, and error patterns also matter.

13. Prediction on New Text

After selecting the best-performing model, we can use it to classify previously unseen text.

For example:

Input:

"The national cricket team secured an impressive victory in the final match."

Expected category:

```
Sport
```

The new article must undergo the same preprocessing and feature extraction steps used during model training.

Python implementation:

```
def predict_news_category(article):
    cleaned_article = clean_text(article)
    tokens = cleaned_article.split()
    filtered_tokens = remove_stopwords(tokens)
    processed_article = " ".join(filtered_tokens)
    article_vector = tfidf_vectorizer.transform(
        [processed_article]
    )
    prediction = best_model.predict(article_vector)
    return prediction[0]
```

Example:

```
sample_article = """
The company announced a major investment
in its new technology division.
"""
```

```
print(predict_news_category(sample_article))
```

The model returns the predicted news category.

14. Model Saving and Loading

Training a machine learning model repeatedly can be time-consuming.

We can save the trained model and the fitted vectorizer for future use.

The model and vectorizer must be saved together because the classifier expects the same feature representation used during training.

Python implementation:

```
import joblib
```

```
joblib.dump(best_model, "bbc_news_model.pkl")
```

```
joblib.dump(
    tfidf_vectorizer,
    "bbc_tfidf_vectorizer.pkl"
)
```

To load them later:

```
loaded_model = joblib.load("bbc_news_model.pkl")
```

```
loaded_vectorizer = joblib.load(
    "bbc_tfidf_vectorizer.pkl"
)
```

These saved artifacts can be integrated into an application or prediction interface.

Only load trusted model files, and maintain compatible library versions.

15. Deployment and Monitoring

The final stage is making the NLP application available for practical use.

A trained model can be integrated into:

- A Flask or FastAPI application
- A Streamlit web application
- A website or internal dashboard
- A larger data processing system

However, deployment is not the end of the workflow.

Real-world language changes over time. New words, topics, writing styles, and trends can affect model performance.

Therefore, a production NLP system should be monitored regularly.

Important activities include:

- Monitoring prediction quality.
- Collecting new data.
- Identifying incorrect predictions.
- Updating preprocessing when necessary.
- Retraining and evaluating models.
- Maintaining model versions.

This creates a continuous improvement cycle.

16. Common Challenges in NLP

While developing NLP applications, we may encounter several challenges.

1. Noisy Text Data

Text may contain spelling mistakes, emojis, abbreviations, and unnecessary characters.

2. Vocabulary Differences

Different words can express similar meanings, making classification difficult.

3. Ambiguous Language

A word can have different meanings depending on its context.

4. Imbalanced Datasets

Some categories may contain significantly more examples than others.

5. High-Dimensional Features

Text vectorization can generate thousands of features, increasing computational requirements.

6. Data Leakage

Using information from the test dataset during training can produce misleading evaluation results.

Understanding these challenges helps us build more reliable NLP applications.

17. Practical Application: BBC News Article Classification

To understand how these stages work together, I implemented an end-to-end NLP project: **BBC News Article Classification**.

The objective was to automatically classify news articles into five categories:

- Business
- Entertainment

- Politics
- Sport
- Tech

The project involved:

1. Dataset loading and exploration.
2. Text cleaning and preprocessing.
3. Tokenization and stopword removal.
4. Stemming and lemmatization experiments.
5. TF-IDF feature extraction.
6. Training Multinomial Naive Bayes and Logistic Regression.
7. Evaluating both models using classification metrics.
8. Comparing their performance.
9. Predicting categories for new articles.
10. Saving the trained model and TF-IDF vectorizer.

This practical implementation helped me understand how unstructured text can be transformed into numerical representations and used by machine learning algorithms to generate predictions.

18. Conclusion

Natural Language Processing plays an important role in enabling machines to work with human language.

From collecting raw text to deploying a trained model, every stage of the NLP workflow contributes to building an effective solution.

The key takeaway is that successful NLP applications are not built by selecting an algorithm alone. They depend on understanding the data, applying appropriate preprocessing, choosing suitable feature extraction techniques, training models carefully, and evaluating their performance correctly.

As NLP continues to evolve, techniques such as word embeddings, transformer architectures, and large language models are opening new possibilities for understanding and generating human language.

The journey from raw text to intelligent predictions begins with a well-designed NLP workflow.

Key Takeaways

- NLP enables computers to process and understand human language.
- Text preprocessing improves consistency and prepares data for modeling.
- Tokenization and stopword removal help prepare text for analysis.
- TF-IDF converts text into numerical features.
- Machine learning algorithms learn patterns from those features.
- Proper evaluation helps measure how well a model generalizes.
- Deployment and monitoring help maintain a useful NLP application.

References and Further Learning

- Scikit-learn — Working with Text Data
- NLTK Documentation
- Scikit-learn — TfidfVectorizer
- Hugging Face NLP Course

#NaturalLanguageProcessing #NLP #MachineLearning #Python #DataScience #TextClassification #TFIDF #ArtificialIntelligence #LearningByDoing



Follow

Written by Bhuvaneshwari

1 follower · 1 following